

Pytania z OS

Podane na wykładzie.....	3
W którym momencie następuje przełączenie z trybu 16-bitowego do 32-bitowego?	3
Jak działa stronicowanie (jak jest ustalany adres).....	3
Jak wygląda wpis w page directory?	3
Co procesor x86 daje programiście przy pisaniu systemu operacyjnego?	3
Dlaczego w katalogu stron nie ma rozmiaru strony?	3
Dlaczego na pendrive FAT, a nie NTFS?	3
Jak zbudowany jest FAT?	3
Działanie FAT. "W katalogu głównym jest plik.txt, jak odczytamy 1 i 3 klaster(jednostkę alokacji pliku)?"	4
Co zawiera rekord plikowy FAT?	4
Co zawiera rekord plikowy NTFS?	4
Wymień kluczowe elementy systemu plików NTFS.....	4
Gdzie znajduje się tablica MFT?	4
Typy atrybutów rekordu plikowego NTFS. Za co odpowiadają atrybuty 10h, 30h, 80h?	4
Podaj system plików z metodą listowo-indeksową dostępu do pliku.	4
Opisz zawartość struktury bio.	4
Opisz budowę i-węzła.	5
Oblicz w którym bloku bezpośrednim, pojedynczo pośrednim czy podwójnie pośrednim znajduje się dany bajt.	5
Pojawiły się na wykładzie, ale nie było powiedziane, że będą.....	6
Jakie informacje są zapisane w rejestrze CR0?	6
Skąd dysk wie, gdzie jest jego pierwszy sektor?	6
Po co w atrybutach rekordu plikowego w FAT rozmiar pliku?.....	6
Po co w katalogu lokalizacja katalogu nadrzędnego?	6
Gdzie można ukryć pliki na dysku?	6
Różnice GPT vs MBR	6
Jak wystartować system z GPT?	6
Wspólne elementy FAT, NTFS, ext (z czego powinien składać się dowolny system plików).....	6
Jak jest zbudowane i jaka jest rola MFT?	6
Co zawiera atrybut \$DATA rekordu plikowego NTFS?	6
Co zawiera katalog w NTFS?.....	6
Co to jest superblok w ext3?	6
Zeszlóroczne	7
Wyjaśnij różnicę między liniowym a fizycznym adresem w pamięci operacyjnej.....	7
Znaczenie rejestru CR3 w procesorach x86	7
Omówić przekształcenie adresu dla 64GB pamięci w architekturze x86	7
Różnica wyznaczania adresu w trybie chronionym z PAE i bez.....	7
Gdzie w zarządzaniu pamięcią użyto metody rejestru bazowego i rejestru granicznego?	7

Coś o MBR i VBR i czy każdy nośnik danych je zawiera.	7
Jak wygląda blok MBR na dysku.....	7
Wymień 3 główne bloki związane z partycjonowaniem dysku w EFI.....	7
Safety w EFI, podać mechanizmy, dzięki którym EFI jest bezpieczniejszy niż BIOS.	7
Piszemy program pod NTFSem, przechowuje w pamięci 1GB plik przyklad.txt - jak się dostać do pierwszego klastra (albo adresu pierwszego klastra?)	8
Dlaczego wprowadzono strukturę bio.....	8
Deadline I/O scheduler , starving reads, opisać zasadę działania.....	8
Dlaczego CFQ Scheduler jest taki dobry?	8
Inne (prawdopodobnie pojawiły się w latach wcześniejszych).	9
Omów proces uruchamiania komputera (od wciśnięcia przycisku zasilania do załadowania systemu operacyjnego).	9
Przedstaw kolejne kroki w komunikacji pomiędzy aplikacją użytkową a sprzętem w Windows.....	9
Wyjaśnij następujące pojęcia MBR, VBR, IDT, GDTR.....	9
W jaki sposób w architekturze IA-32 została zrealizowana koncepcja rejestru bazowego i granicznego jako metody izolacji procesów?	9
Ile stosów ma procedura w trybie chronionym? Wyjaśnij dlaczego tak się dzieje.	9
Opisz mechanizm obsługi przerwania na poziomie jądra w systemie Windows XP.	10
Dlaczego w trybie chronionym x86 zostały wyróżnione deskryptory bramy, a nie zostały wykorzystane deskryptory przerwania?	10
Jakie zalety ma realizacja pamięci wirtualnej za pomocą stronicowania w porównaniu z realizacją za pomocą segmentacji?	10
Omów typowe działanie systemu operacyjnego przy błędach strony (page fault handling).	10
Podaj cele stosowania struktury GDT.....	10
Przedstaw charakterystykę systemów opartych o EFI.....	10
Omówić implementację pamięci wirtualnej za pomocą segmentacji	11
Omówić strukturę partycji rozszerzonej na dysku.....	11
Omów mechanizm journalingu. Wymień nazwy systemów plików, w których jest stosowany.	11
W jaki sposób tablica alokacji plików (FAT) opisuje położenie plików na dysku?	11
Wymień 5 różnych systemów plików.....	11
Porównaj poznane schedulery dla urządzeń blokowych I/O w systemie Linux.	12

Podane na wykładzie.

W którym momencie następuje przełączenie z trybu 16-bitowego do 32-bitowego?

W boot loaderze - np. winload.exe.

Jak działa stronicowanie (jak jest ustalany adres).

Najpierw wyznaczany jest adres liniowy na podstawie sumy adresu efektywnego(offsetu) i adresu bazowego odczytanego z deskryptora segmentu znajdującego się w tablicy GDT lub LDT, na który wskazuje zawartość rejestru segmentowego (selektor), a dokładniej jego 13 najstarszych bitów. Na położenie tablicy GDT wskazuje rejestr GDTR, a tablicy LDT - LDTR. To czy korzystamy z GDT czy LDT określa jeden z bitów selektora.

Na lokalizację katalogu stron danego procesu wskazuje rejestr CR3.

Najstarsze 10 bitów adresu liniowego wskazuje na wpis (PDE - page directory entry) znajdujący się w katalogu stron. Z tego wpisu odczytywany jest adres pod którym znajduje się tablica stron. Kolejnych 10 bitów odczytanych z adresu liniowego wskazuje na rekord (PTE - page table entry) z którego odczytywany jest adres fizyczny strony. Najmłodsze 12 bitów adresu liniowego(offset) jest sumowane z adresem fizycznym strony i w ten sposób otrzymujemy adres fizyczny szukanej komórki pamięci.

Jeżeli nie jest używane PAE, to adresy pod którymi znajduje się PD, PT i strona pamięci mają 32 bity. Jeżeli PAE jest wykorzystywane to adresy te mają 52 bity. Dodawany jest również nowy "katalog" PDPT (page directory pointer table), na który teraz wskazuje rejestr CR3, a z kolei wpis z tego katalogu wskazuje dopiero na odpowiedni katalog stron. Do PDPT (page directory pointer table entry) odwołujemy się za pomocą 2 najstarszych bitów z adresu liniowego, w konsekwencji wcześniej używane 10bitowe części wskazujące na PDE i PTE są zmniejszone do 9 bitów.

Jak wygląda wpis w page directory?

- adres tablicy stron
- bit obecności (czy strona jest w pamięci)
- czy jest to strona systemowa czy użytkownika
- czy może być cache'owana
- bit write-through
- bit uprawnienia do zapisu
- bit informujący czy strona jest brudna (czy była modyfikowana)

Co procesor x86 daje programiście przy pisaniu systemu operacyjnego?

- segmentację
- stronicowanie
- poziomy bezpieczeństwa
- ochronę pamięci
- wspiera przerwania
- wsparcie przy przełączaniu zadań

Dlaczego w katalogu stron nie ma rozmiaru strony?

Rozmiar strony jest stały (zwykle 4kB), więc nie ma konieczności dodatkowo umieszczać go w katalogu stron.

Dlaczego na pendrive FAT, a nie NTFS?

FAT ma mniejszy narzut danych opisujących strukturę systemu plików.

Jak zbudowany jest FAT?

- VBR (w FAT32 w nim jest zapisana lokalizacja katalogu głównego)
- FAT
- kopia FAT
- katalog główny (w FAT32 obszar danych od razu po kopii FAT, a katalog główny gdzieś w tym obszarze danych)
- obszar danych

Działanie FAT. "W katalogu głównym jest plik.txt, jak odczytamy 1 i 3 klaster(jednostkę alokacji) pliku?"

1. Odczytujemy z katalogu głównego rekord plikowy pliku plik.txt.
2. Odczytujemy z rekordu plikowego pozycję początkową pliku w obszarze danych, liczba ta jest jednocześnie numerem pozycji w tablicy FAT z której odczytamy lokalizację następnego klastra.
3. Odczytujemy pierwszy klaster pliku.
4. Odczytujemy z tablicy FAT pozycję drugiego klastra, jak wcześniej wspomniano, liczba ta jest również numerem następnej z kolei pozycji w tablicy FAT.
5. Odczytujemy z tablicy FAT pozycję trzeciego klastra.
6. Odczytujemy trzeci klaster.

Co zawiera rekord plikowy FAT?

- krótka nazwa pliku
- atrybuty pliku
- nr początkowej pozycji FAT32
- rozmiar pliku
- czas, data utworzenia
- czas, data ostatniej modyfikacji

Co zawiera rekord plikowy NTFS?

- nagłówek
- atrybuty w postaci: nazwa, wartość
- znacznik końca rekordu FFFFFFFFh

Wymień kluczowe elementy systemu plików NTFS.

- sektor ładujący
- MFT
- pliki systemowe
- obszar danych

Gdzie znajduje się tablica MFT?

Pierwsza część zawsze w jednym, z góry zdefiniowanym miejscu (w obszarze plików systemowych). Reszta jej fragmentów, o ile występują, jest rozsiana po obszarze danych.

Typy atrybutów rekordu plikowego NTFS. Za co odpowiadają atrybuty 10h, 30h, 80h?

- rezydentne - w całości w MFT
- nierezydentne - część w MFT, część poza (np. dane)
- stałym rozmiarze
- zmiennym rozmiarze (mają pole opisujące rozmiar)

Z kombinacji 4 powyższych mamy 4 typy atrybutów. Każdy atrybut składa się z pary: nazwa, wartość.

Wybrane numery atrybutów:

- 10h - informacje standardowe
- 30h - nazwa
- 80h - dane

Podaj system plików z metodą listowo-indeksową dostępu do pliku.

NTFS

Opisz zawartość struktury bio.

Reprezentuje aktywne operacje i/o w postaci listy ciągłych obszarów w pamięci (segmentów). Bufory nie muszą zajmować ciągłej przestrzeni w pamięci (scatter-gather I/O). Jej głównymi elementami są:

- bi_io_vec - tablica wskaźników na bio_vec (wektor opisujący pojedynczy segment)
- licznik użycia (ile procesów z niej korzysta)
- flagi związane z operacją i/o

W jej skład nie wchodzi informacje statystyczne opisujące bufor, tak jak było to w przypadku buffer_head.

Opisz budowę i-węzła.

Każdy i-węzeł zawiera:

- Tablicę adresów, za pomocą której można zlokalizować części pliku na dysku
- Rozmiar pliku
- Identyfikator właściciela pliku
- Typ pliku
- Prawa dostępu do pliku
- Czas dostępu/modyfikacji pliku
- Liczbę dowiązań do pliku
- Liczbę procesów, które otworzyły dany plik

i-węzeł nie zawiera nazwy pliku!

Oblicz w którym bloku bezpośrednim, pojedynczo pośrednim czy podwójnie pośrednim znajduje się dany bajt.

- 12 bloków bezpośrednich po 4KB = 48KB

- 1 blok pośredni i 1024 bloki bezpośrednie = 4MB

- 1 blok podwójnie pośredni i 1024 pośrednie = 4GB

- 1 blok potrójnie pośredni i 1024 bloki podwójnie pośrednie = 4TB

• Bajt 30000 to $30000/4096 = 7$ reszta 1328

– Potrzebny bajt znajduje się w bloku, którego numer podany jest na pozycji 7 tablicy adresów bloków,

– Wewnątrz bloku żądany bajt jest odległy o 1328 bajtów od początku bloku

• bajt 5 000 000

– Wymaga użycia bloku podwójnie pośredniego

– Pierwszy bajt dostępny jest poprzez blok podwójnie pośredni i ma numer $48KB + 4MB = 49152 + 4194304 = 4\,243\,456$

– Bajt o numerze 5 000 000 jest więc bajtem o numerze 756 544 względem początku bloku podwójnie pośredniego

– Każdy blok pojedynczo pośredni daje dostęp do 4MB, więc bajt o numerze 5 000 000 będzie wskazywany pośredni przez pozycję zerową bloku podwójnie pośredniego – po odczytaniu wyznaczonego w ten sposób bloku pośredniego poszukuje się w nim pozycji o indeksie $756\,544/4096 = 184$ (reszta 2880)

– Zawartość pozycji o indeksie 184 wskazuje na blok dyskowy zawierający żądany bajt – jest on odległy o 2880 bajtów od początku bloku

Pojawiły się na wykładzie, ale nie było powiedziane, że będą.

Jakie informacje są zapisane w rejestrze CR0?

Między innymi czy włączone jest stronicowanie i czy włączona jest ochrona stron systemowych.

Skąd dysk wie, gdzie jest jego pierwszy sektor?

Sektory na dysku rozpoczynają się od preambuły, potem są pewne metadane (np. numer bloku), dane, a na końcu suma kontrolna. W metadanych znajduje się informacja czy jest to sektor startowy, głowica przesuwa się nad odpowiednią ścieżkę i odczytuje sektory do momentu aż natrafi na sektor startowy.

Po co w atrybutach rekordu plikowego w FAT rozmiar pliku?

Bo czytamy zawsze jeden klastr i jak plik nie wypełnia w całości ostatniego klastra to musimy wiedzieć ile z niego obciąć.

Po co w katalogu lokalizacja katalogu nadrzędnego?

Żeby można było się łatwo cofnąć, bez przeszukiwania od początku.

Gdzie można ukryć pliki na dysku?

- w extended MBR
- w niepełnych końcówkach klastrów
- w niezajętym przez partycje miejscu

Różnice GPT vs MBR

- Poszerzenie LBA do 64 bitów
- Powiększenie maksymalnego rozmiaru partycji, ilości partycji
- Zmiany w obszarze Safety & Security

Jak wystartować system z GPT?

Kod startowy jest w firmwarze EFI, przy instalacji systemu do zmiennych systemowych EFI przekazywany jest GUID partycji startowej i lokalizacja pliku boot managera.

Wspólne elementy FAT, NTFS, ext (z czego powinien składać się dowolny system plików)

- sektor ładujący
- tablica z indeksami (lokalizacjami) plików w obszarze danych
- obszar danych

Jak jest zbudowane i jaka jest rola MFT?

MFT jest tablicą zawierającą wpisy opisujące każdy plik/katalog znajdujący się na partycji NTFS w postaci rekordu plikowego o zmiennej długości (1024 - 4096 bajtów).

MFT składa się z:

1. plik opisujący MFT
2. fragment MFT (kopia)
3. pliki systemowe NTFS (pliki metadanych)
4. pliki i katalogi użytkownika

Co zawiera atrybut \$DATA rekordu plikowego NTFS?

Jeżeli plik jest mały to bezpośrednio dane tego pliku, jak jest duży to listę dowiązań do obszaru danych.

Co zawiera katalog w NTFS?

B-drzewo zawierające odnośniki do plików.

Co to jest superblok w ext3?

Superblok to struktura opisująca właściwości i zawierająca dane statystyczne związane z obsługą danej partycji. W różnych lokalizacjach mogą znajdować się kopie (backupy) superbloku.

Zesłoroczne

Wyjaśnij różnicę między liniowym a fizycznym adresem w pamięci operacyjnej.

Adres fizyczny wskazuje bezpośrednio na położenie komórki pamięci w fizycznej przestrzeni adresowej.

Adres liniowy wskazuje na położenie komórki w przestrzeni adresów liniowych (wirtualnych).

W przypadku wyłączenia stronicowania, adres liniowy jest tożsamy adresowi fizycznemu. W przeciwnym wypadku jest on przekształcany na adres fizyczny za pomocą serii operacji wykonywanych przez procesor i system operacyjny.

Znaczenie rejestru CR3 w procesorach x86

Wskazuje lokalizację katalogu stron w pamięci, każdy proces posiada swój własny katalog stron, a więc też ma własną wartość rejestru CR3 - wartość w nim zapisana ulega zmianie przy przełączaniu procesów. Pozwala na rozdzielenie przestrzeni adresowych różnych procesów.

Omówić przekształcenie adresu dla 64GB pamięci w architekturze x86

Żeby zaadresować 64GB pamięci potrzeba co najmniej 36-bitowej szyny adresowej. W związku z tym niezbędne jest PAE. Przebieg wyznaczania adresu jest opisany w punkcie "Jak działa stronicowanie (jak jest ustalany adres)".

Różnica wyznaczania adresu w trybie chronionym z PAE i bez.

Bez PAE mamy 32bitowy adres bazowy ramki strony, z PAE mamy 52 bitowy (PDE i PTE są odpowiednio dłuższe). W PAE dodany zostaje kolejny "katalog" PDPT (page directory pointer table) zawierający adresy katalogów stron, konkretny wpis (PDPTPE) jest wskazywany przez 2 najstarsze bity adresu liniowego. Liczba bitów wskazująca na indeksy w PD i PT zostaje zmniejszona z 10 do 9. Rejestr CR3 wskazuje na PDPT, a nie jak wcześniej na PD. 12 najmłodszych bitów adresu liniowego w obu przypadkach służy jako offset dodawany do adresu fizycznego ramki.

Podział adresu liniowego: z PAE 2 | 9 | 9 | 12, bez PAE 10 | 10 | 12.

Gdzie w zarządzaniu pamięcią użyto metody rejestru bazowego i rejestru granicznego?

Przy segmentacji, w deskryptorze segmentu znajdującym się w GDT.

Coś o MBR i VBR i czy każdy nośnik danych je zawiera.

MBR i VBR są podobnie zbudowane.

W MBR znajduje się lista partycji i kod mający za zadanie przejrzeć listę partycji i załadować sektor ładujący odpowiedniego VBRa.

VBR zawiera między innymi informacje o systemie plików na danej partycji i rozkaz skoku do miejsca, w którym znajduje się bootloader systemu operacyjnego.

Urządzenia przenośne, mające jedną partycję (np. pendrive, płyta CD) mają tylko VBR.

Jak wygląda blok MBR na dysku.

MBR umieszczony jest na dysku w miejscu C:0 H:0 S:1 (LBA0). Na samym początku znajduje się master boot code, czyli krótki kod mający za zadanie przejrzeć listę partycji i załadować w swoje miejsce w pamięci odpowiedni sektor ładujący z partycji aktywnej. Następna, przesunięta o 1BEh, jest w nim 64 bajtowa tablica partycji, zawierająca cztery 16 bajtowe wpisy. W każdym wpisie jest informacja gdzie znajduje się początek odpowiadającej mu partycji. Ostatnie dwa bajty zajmuje kod AA55h identyfikujący rekord MBR.

Wymień 3 główne bloki związane z partycjonowaniem dysku w EFI.

- Protective MBR zapisany w LBA0 - chroni dysk przed nadpisaniem w przypadku podłączenia do systemu nie opartego o EFI
- Primary GPT
- (?) Partycje
- Secondary GPT

Safety w EFI, podać mechanizmy, dzięki którym EFI jest bezpieczniejszy niż BIOS.

- pracuje w trybie chronionym
- podmienić kod ładujący w MBR jest stosunkowo prosto, w GPT jest to znacznie utrudnione
- w GPT mamy kopię nagłówka, w MBR w przypadku uszkodzenia nie da się go odtworzyć
- nagłówek GPT jest chroniony przez sumę kontrolną, w wypadku jego modyfikacji można to wykryć, w MBR nie da się

Piszemy program pod NTFSem, przechowuje w pamięci 1GB plik przyklad.txt - jak się dostać do pierwszego klastra (albo adresu pierwszego klastra?)

Algorytm ogólny:

1. Odszukamy w MFT rekord plikowy pliku przyklad.txt.
2. Plik jest zbyt duży żeby zmieścić się bezpośrednio w atrybucie dane, więc znajduje się tam tablica dowiązań do klastrów w postaci VCN | LCN | liczba kl.
3. Odszukujemy wiersz z wpisem w kolumnie VCN (virtual cluster number) odpowiadającym szukanemu klastrowi (może nie być bezpośredniego dowiązania, wtedy interesuje nas najbliższy numer mniejszy od szukanego).
4. Odczytujemy LCN, jeżeli VCN nie wskazywał bezpośrednio na szukany klaster to dodajemy do odczytanej liczby odpowiednią wartość.

Dlaczego wprowadzono strukturę bio.

Struktura bio rozwiązuje wiele problemów, które występowały w poprzedzającej ją strukturze buffer_head.

- Dzięki jej organizacji bufor nie muszą zajmować ciągłego miejsca w pamięci, wpłynęło to na ograniczenie jej fragmentacji.
- Zawiera tylko informacje niezbędne do przeprowadzenia odczytu/zapisu, jest lekką strukturą.
- Opis bufora w postaci segmentów pozwala Kernelowi wykonywać operacje blokowe I/O na pojedynczym buforze rozproszonym w pamięci, nie wymaga zbędnego podziału operacji I/O (scatter-gather I/O).

Deadline I/O scheduler , starving reads, opisać zasadę działania

write starving reads - problem związany z kolejkowaniem żądań IO przez linux elevator. Zapis jest asynchroniczny, odczyt synchroniczny. Zapisy w jednej części dysku blokują odczyty w innej (aplikacja czeka na dane).

Deadline I/O scheduler

- 3 kolejki żądań:
 - kolejka posortowana po sektorach
 - kolejka dla odczytów
 - kolejka dla zapisów
- żądania oznaczone czasem wygaśnięcia(0.5s dla read, 5s dla write), wedle którego są posortowane w kolejkach read/write
- żądanie do realizacji wybierane z jednej z trzech kolejek na podstawie algorytmu round robin
- problem "write starving reads" nie występuje

Dlaczego CFQ Scheduler jest taki dobry?

Bo każdy proces posiada osobną kolejkę żądań, są one pobierane algorytmem round robin. Dzięki temu wszystkie procesy są traktowane na równi, proces generujący bardzo dużą ilość żądań nie wpłynie negatywnie na wydajność pozostałych procesów w systemie.

Inne (prawdopodobnie pojawiły się w latach wcześniejszych).

Omów proces uruchamiania komputera (od wciśnięcia przycisku zasilania do załadowania systemu operacyjnego).

BIOS

Po włączaniu komputera z adresu F000:FFF0 znajdującego się w ROMie BIOSu ładowany jest rozkaz skoku do lokalizacji pamięci pod którą znajduje się kod POST. Następnie przeglądana jest lista urządzeń w poszukiwaniu tego, które zawiera MBR. MBR zawiera kod odpowiedzialny za przeglądnięcie listy partycji i zlokalizowanie partycji aktywnej zawierającej VBR(czyli takiej, na której znajduje się system operacyjny). VBR lokalizuje i inicjuje wykonanie boot managera.

EFI

W EFI pominięte są etapy związane z MBR i VBR, po wykonaniu POST następuje przejście bezpośrednio do boot managera. Zestaw zmiennych środowiskowych w EFI zawiera dokładną ścieżkę urządzenia i lokalizację pliku boot managera.

Kod boot managera wywołuje boot loader wybranego systemu operacyjnego. W boot loaderze następuje załadowanie niezbędnych do rozruchu elementów systemu (kernel, HAL, sterowniki) i przełączenie z trybu rzeczywistego do chronionego (w przypadku BIOS). Sterowanie zostaje przekazane do kernela, więc dalszy proces rozruchu systemu nie jest już zależny od BIOS/EFI. Ładowane są podstawowe usługi i sterowniki, a następnie menadżer sesji(smss.exe), którego zadaniem jest przygotowanie środowiska dla użytkownika. Ładuje on najczęściej wykorzystywane DLLki i bibliotekę obsługującą API systemowe wystawiane dla aplikacji użytkownika. Następnie tworzy on swoje dwie kopie: sesja 0 jest sesją systemową odpowiedzialną za załadowanie wymaganych przez pozostałe sesje podsystemów, natomiast sesja 1 (i ewentualnie kolejne) służy wyświetleniu interfejsu umożliwiającego załogowanie się użytkownika.

Komunikacja pomiędzy sesjami odbywa się za pośrednictwem csrss.exe (client-server runtime subsystem). W przypadku zamykania systemu komunikat jest przekazywany do najstarszej instancji smss.exe (tej, która podzieliła się tworząc swoje dzieci) i to ona zamyka system.

Przedstaw kolejne kroki w komunikacji pomiędzy aplikacją użytkową a sprzętem w Windows.

Aplikacja użytkownika wywołuje funkcję z Windows API, odwzorowywaną na funkcje z ntdll.dll. Następnie musi zostać wywołana funkcja na poziomie kernela (wyższy poziom uprzywilejowania). Odbywa się to za pomocą mechanizmu deskryptora bram. Żądanie trafia do odpowiedniego sterownika a z niego do warstwy HAL (hal.dll), która już ogarnia komunikację ze sprzętem.

Wyjaśnij następujące pojęcia MBR, VBR, IDT, GDTR.

MBR – Master Boot Record. Jest to pierwszy sektor na dysku zawierający specjalny program, który jest odczytywany przy starcie komputera do pamięci RAM i którego zadaniem jest przeszukanie tablicy partycji w celu przejścia do VBR.

VBR – Volume Boot Record. Podobny do MBR, ale znajduje się już na konkretnej partycji (na początku) i zawiera program, który powinien rozpocząć ładowanie systemu operacyjnego.

IDT – Interrupt Descriptor Table. Jest to tablica deskryptorów znajdująca się w pamięci RAM, w której zapisane są selektory, które z kolei mają indeksować tablicę GDT, która wskazuje segment, w jakim znajduje się procedura obsługi przerwania.

GDTR – Global Descriptor Table Register. Jest to 48-bitowy rejestr wskazujący położenie w pamięci tablicy GDT (32 bity) oraz limit rozmiaru tej tablicy (16 bitów).

W jaki sposób w architekturze IA-32 została zrealizowana koncepcja rejestru bazowego i granicznego jako metody izolacji procesów?

Rejestry ograniczają dostęp procesu do pamięci operacyjnej określając dopuszczalny zakres adresów, do których może odwoływać się proces. Rejestr bazowy przechowuje najmniejszy dopuszczalny adres fizyczny pamięci, a rejestr graniczny zawiera rozmiar obszaru pamięci, obie te informacje są zapisane w odpowiednich polach deskryptora segmentu. W przypadku odwołania się do pamięci poza wyznaczonym obszarem procesor wygeneruje wyjątek.

Ile stosów ma procedura w trybie chronionym? Wyjaśnij dlaczego tak się dzieje.

Dana procedura wykonywana jest przez wątek. Każdy wątek posiada dwa stosy – po jednym w trybie jądra i użytkownika. Powodem tego jest bezpieczeństwo – nic co działa w trybie użytkownika nie ma dostępu do danych z trybu jądra.

Opisz mechanizm obsługi przerwania na poziomie jądra w systemie Windows XP.

Przerwania w systemie Windows są realizowane poprzez wirtualizację obsługi przerwania (przerwania od sprzętu są dostarczane w odpowiedniej formie z warstwy HAL). Po wystąpieniu przerwania odczytywana jest z rejestru IDTR lokalizacja IDT (interrupt descriptor table). Następnie z odpowiedniego deskryptora przerwania odczytywany jest segment selector (ładowany do rejestru CS) i offset. Teraz można odczytać kod obsługi przerwania z GDT.

Dlaczego w trybie chronionym x86 zostały wyróżnione deskryptory bramy, a nie zostały wykorzystane deskryptory przerwania?

Żeby uniezależnić się od architektury. Np. windows oszukuje i wszystkie deskryptory przerwania wskazują w jedno miejsce. Windowsy budują własną strukturę obsługi przerwania, przerwanie sprzętowe są mapowane przez HAL. W ten sposób przy zmianie sprzętu wystarczy wymienić warstwę HAL.

Jakie zalety ma realizacja pamięci wirtualnej za pomocą stronicowania w porównaniu z realizacją za pomocą segmentacji?

Segmentacja:

- segmenty różnych rozmiarów, powoduje to dużą fragmentację pamięci

Stronicowanie:

- każdy proces ma własną wartość rejestru CR3 (własny katalog stron), zapewnia to separację fizycznych przestrzeni adresowych procesów mimo, że w przestrzeni liniowej mogą być one takie same
- pojedyncza strona jest stałego rozmiaru, zazwyczaj 4kb. Jest to łatwe w zarządzaniu, nie występuje fragmentacja pamięci.
- dzięki wyeliminowaniu fragmentacji wolne miejsce w pamięci zużywa się znacznie wolniej
- gdy poszerzamy pulę pamięci o przestrzeń dyskową stronicowanie działa dużo szybciej bo nie musi ładować dużych bloków
- stronicowanie pozwala na zastosowanie przestrzeni liniowej adresów większej niż fizyczna przestrzeń adresowa, segmentacja nie daje takiej możliwości

Omów typowe działanie systemu operacyjnego przy błędach strony (page fault handling).

System sprawdzi, czy w pamięci znajduje się wolna ramka do której można załadować stronę z dysku, jeżeli tak to załaduje ją. Jeżeli nie ma wolnego miejsca to zostanie wybrana jedna ze stron aktualnie znajdujących się w pamięci i odesłana na dysk, a w jej miejsce wczytana zostanie potrzebna strona. Po umieszczeniu strony w pamięci sterowanie zostanie przekazane do miejsca w którym wystąpił wyjątek braku strony.

Podaj cele stosowania struktury GDT.

- uzyskanie liniowej przestrzeni pamięci, niezależnie od fizycznego rozproszenia danych
- umożliwienie obsługi większych ilości pamięci jeżeli ograniczał nas wcześniej rozmiar rejestru (np. szyna jest 20 bitowa, ale rejestry 16 bitowe)
- zwiększenie funkcjonalności i kontroli nad tym, co się dzieje - dodanie opisów segmentów – kod/dane, opisy przywilejów, czy segment jest w pamięci
- zwiększenie bezpieczeństwa (ochrona pamięci - rejestr bazowy i graniczny)

Przedstaw charakterystykę systemów opartych o EFI.

- są niezależne od implementacji sprzętu
- nie wymagają trybu rzeczywistego procesora do rozruchu
- nie nakładają limitów na całkowity rozmiar pamięci ROM. Sterowniki i aplikacje mogą być ładowane z dowolnego źródła w dowolne miejsce przestrzeni adresowej.
- Zawierają dodatkową powłokę (EFI shell)
- Ewoluuje od VGA do UGA (Universal Graphics Adapter)
- partycjonowanie GPT, obsługują dyski >2TB

Omówić implementację pamięci wirtualnej za pomocą segmentacji .

Polega na przechowywaniu niektórych segmentów programu w pamięci dyskowej i sprowadzaniu ich do pamięci operacyjnej, gdy stają się potrzebne.

Obecność segmentu w pamięci operacyjnej wskazuje bit P(present). Jeżeli wystąpi odwołanie do deskryptora z $P = 0$ to pojawia się wyjątek(przerwanie procesora), w takiej sytuacji system odszukuje wolny obszar pamięci (lub ją zwalnia), pobiera potrzebny segment do pamięci i powraca do realizowanego programu.

Występuje również bit W oznaczający czy segment jest tylko do odczytu ($W=0$), wszystkie ładowane do pamięci segmenty są wstępnie oznaczane jako tylko do odczytu. Jeżeli wystąpi zapis to procesor wygeneruje wyjątek, którego program obsługi sprawdzi legalność zapisu, ustawi $W=1$ i przekaże sterowanie z powrotem. Jeżeli segment ma zostać przeniesiony do pamięci dyskowej to sprawdzany jest stan tego bitu i w przypadku, gdy nie był on modyfikowany operacja zapisu na dysk nie jest potrzebna.

Bit A przechowuje informacje statystyczne związane z wykorzystaniem segmentów, potrzebne do ich racjonalnej wymiany. Ustawiany jest na 1, gdy do rejestru segmentowego zostanie wpisany selektor wskazujący rozpatrywany deskryptor. Bit A nie jest nigdy automatycznie zerowany przez procesor, system operacyjny zeruje bity A w obserwowanych segmentach i następnie okresowo odczytuje ich zawartość każdorazowo wykonując te zerowanie. Jeśli dla pewnego segmentu, kilkakrotnie odczytano zerową wartość bitu A, to segment ten jest zapewne nieużywany.

Adres fizyczny komórki pamięci jest wyznaczany na podstawie sumy adresu efektywnego(offsetu) i adresu bazowego. W trybie rzeczywistym adres ten jest odczytywany bezpośrednio z rejestru, a w chronionym z deskryptora segmentu znajdującego się w tablicy GDT lub LDT, na który wskazuje zawartość rejestru segmentowego (selektor).

Omówić strukturę partycji rozszerzonej na dysku.

Na początku partycji rozszerzonej znajduje się sektor o specyficznej strukturze, w którym zawarta jest tablica partycji, położona w tym samym miejscu, co w przypadku MBR'a. W tablicy tej wykorzystywane są tylko dwie pierwsze pozycje. Pierwszy element tablicy partycji wskazuje adres początkowy dysku logicznego.

Drugi – sektor dyskowy o takiej samej strukturze, stanowiący kolejny element listy dysków logicznych.

Element zawierający 0 w polu identyfikatora systemu operacyjnego wskazuje koniec tej listy.

Omów mechanizm journalingu. Wymień nazwy systemów plików, w których jest stosowany.

Przy użyciu księgowania dane nie są od razu zapisywane na dysk, tylko operacje do wykonania na nich są zapisywane w swoistym dzienniku/kronice (ang. journal). Dzięki takiemu mechanizmowi działania zmniejsza się prawdopodobieństwo utraty danych: jeśli utrata zasilania nastąpiła w trakcie zapisu - zapis zostanie dokończony po przywróceniu zasilania, jeśli przed - stracimy tylko ostatnio naniesione poprawki, a oryginalny plik pozostanie nietknięty. Jest to rodzaj operacji transakcyjnych podobnych do tych stosowanych w DBMSach.

Systemami plików z księgowaniem są: ext3, ext4, NTFS, HFS+ (system plików Apple dla Mac OS).

W jaki sposób tablica alokacji plików (FAT) opisuje położenie plików na dysku?

File allocation table (FAT) zawiera informacje o stanie wszystkich klastrów obszaru danych dysku. Każda pozycja tablicy używana przez plik wskazuje następną pozycję w FAT dla tego pliku (metoda listowa). Sekwencja numerów kolejnych pozycji jest taka sama jak numery klastrów składających się na cały plik. Pierwsze dwie pozycje w tablicy są zarezerwowane. Trzecia zawiera informację o pierwszym klastrze obszaru danych.

Wymień 5 różnych systemów plików.

fat32, reiserfs, ntfs, xfs, ext3

Porównaj poznane schedulery dla urządzeń blokowych I/O w systemie Linux.

Linus Elevator

- używa prostego algorytmu grupowania żądań
 - gdy w kolejce znajduje się żądanie do przyległego sektora dysku, żądania te są scalane,
 - gdy żądanie znajdujące się w kolejce jest w niej już długo, nowe żądanie jest umieszczane na końcu kolejki – w celu uniknięcia zagłodzenia starszego żądania,
 - gdy w kolejce znajdowane są żądania do bliskich sektorów nieprzylegających do siebie ani do sektorów obsługiwanych przez nowe żądanie – jest ono umieszczane pomiędzy tymi żdaniami z kolejki
 - gdy nie zachodzą powyższe reguły, żądanie umieszczane jest na końcu kolejki
- wspólna kolejka dla odczytów i zapisów
- intensywność operacji wykonywanych na jednej części dysku może zagłodzić inne operacje
- występuje problem "write starving reads"

Deadline I/O scheduler

- 3 kolejki żądań:
 - kolejka posortowana po sektorach
 - kolejka dla odczytów
 - kolejka dla zapisów
- żądania oznaczone czasem wygaśnięcia, po którym są posortowane w kolejkach read/write
- żądanie do realizacji wybierane z jednej z trzech kolejek na podstawie algorytmu round robin
- problem "write starving reads" nie występuje

Anticipatory I/O scheduler

- Heurystycznie przewiduje operacje, które mogą zaistnieć na podstawie statystyk dla każdego z procesów.
- Bardzo dobrze zachowuje się w przewidywalnym środowisku, gdzie nie występuje ingerencja użytkownika (np. serwery, systemy wbudowane)
- problem "write starving reads" nie występuje

CFQ I/O scheduler

- osobna kolejka żądań dla każdego procesu
- pobieranie kolejnych żądań algorytmem round robin
- uczciwa obsługa każdego z procesów - proces generujący dużo żądań jest taktowany na równi z procesem, który generuje ich mniej

Noop I/O scheduler

- brak sortowania
- dobry dla urządzeń o losowym dostępie (np. kart pamięci flash)